

模拟赛 1 讲解

meow

Cap1taL

2026 年 8 月

题意

对于所有位置，找到尽可能大的正方形，使得内部颜色种类数不超过 k
 $n \leq 500, k, a_{i,j} \leq n$

Solution

- 考虑暴力做法，直接枚举所有正方形，然后遍历正方形维护颜色数量，复杂度为 $O(n^5)$ ，固定端点后精细实现可以做到 $O(n^4)$ ，可以通过 $n \leq 50$ ，此处不再赘述
- 考虑 $a \leq 2$ ，只需要进一步讨论 $k = 1$ 和 $k \geq 2$ ，前者需要找同色最大正方形，后者则总是最大的正方形
- 考虑 $k = 2$ ，会发现等价于我们固定住端点扩展长度，当“出现第三个颜色”时非法
- 用各类方法实现 $k = 2$ 后，会自然引导我们尝试刻画“每个颜色什么时候出现”

Solution

- 考虑正解，观察到值域并不大，我们可以接受与值域有关的复杂度
- 先对每个位置、每种颜色处理 $f_{i,j,c}$ 表示以 (i,j) 为左上角，边长多长时第一次出现 c 这个颜色。可以从 $(i+1,j+1), (i,j+1), (i+1,j)$ 转移到 (i,j) 。
- 有了 f 之后，我们可以进一步求出 $g_{i,j,L}$ 表示有多少个 c 满足 $f_{i,j,c} = L$ 。可以直接用 f 计算，这一部分是简单的。
- 基于此，我们就可以枚举每个左上角端点 (i,j) ，然后线性扫描 $g_{i,j,L}$ ，使得前缀和不超过 k 的最大 L 即为答案。复杂度 $O(n^3)$
- 具体实现上，开两个 $O(n^3)$ 的数组可能超过内存限制。可以考虑将 `int` 更换为 `short`，或者更好的做法是不完整存下 $f_{i,j,c}$ ，而是对每个颜色处理出 $f_{i,j}$ 后直接计算 g 数组。

题面

一棵树，有边权，边权为一个排列。要求给无向边定向，使一个所有可能的路径都边权递减，问定向方案数

$$n \leq 3 \times 10^5$$

Solution

- $n \leq 15$ 的部分分可以直接暴力枚举定向方案，复杂度为指数级
- $n \leq 3000$ 给一些可能的平方做法，或者 dp 实现不精细的做法
- 考虑链，容易发现决策满足很好的无后效性，可以直接做线性 DP，设 $f_{i,0/1}$ 表示 i 左侧的边的朝向，讨论边权大小关心转移即可
- 考虑菊花，此时可能的路径长度只有 2。由于要求路径权值递增，因此不能存在“较大的边朝向中心，较小的边朝向外侧”的情况。
- 更具体的，若我们把菊花图的边按照边权升序排序，那么定向方案一定是前缀朝向中心，后缀远离中心

Solution

- 链和菊花已经给了我们足够多的提示
- 把我们刚才的算法上树，即设 $f_{u,0/1}$ 表示 u 到父亲边为 0/1 的情况下整个子树的方案数
- 转移时模仿菊花图，给每个点所有的边排序，之后用子节点更新父节点即可
- 具体实现时可以把连向父亲的边和与孩子的边一起排序，然后扫描记录 flag 来决定转移到 $f_{u,0}$ 还是 $f_{u,1}$
- 复杂度 $O(n \log n)$ ，瓶颈在给边排序

题面

一个序列，每次随机合并相邻两个数，合并结果是他们的平均数，直到序列长度为 1。求最终结果的期望
 $n \leq 10^6$

Solution

- n 较小的部分留给分类讨论或者暴力枚举的做法
- 观察题目合并的形式，实际上答案形如 $\sum \frac{a_i}{2^{k_i}}$ ，其中 k_i 的数值与合并顺序有关
- 根据期望的线性性，我们有

$$E\left(\sum \frac{a_i}{2^{k_i}}\right) = \sum E\left(\frac{a_i}{2^{k_i}}\right) = \sum a_i \times E\left(\frac{1}{2^{k_i}}\right)$$

- 由此我们成功完成了对期望的拆分，可以来考虑求出每个位置的系数是多少

Solution

求系数

- 因为是在求系数，我们完全不关心 a_i ，所以自然我们只关心左侧和右侧有多少个数
- 这给了我们一个 n^2 DP 的尝试，设 $f_{i,j}$ 表示左侧有 i 个数，右侧有 j 个数，此位置最终系数的期望是多少。那么 $k_i = f_{i-1, n-i}$
- 转移只需要尝试在最左侧或最右侧加一个数 n^2 DP 的尝试，设 $f_{i,j}$ 表示左侧有 i 个数，右侧有 j 个数，此位置最终系数的期望是多少。那么 $k_i = f_{i-1, n-i}$

$$f_{i,j} = \frac{i-1}{i+j} \times f_{i-1,j} + \frac{j-1}{i+j} \times f_{i,j-1} + \frac{1}{i+j} \times \frac{f_{i,j-1}}{2} + \frac{1}{i+j} \times \frac{f_{i-1,j}}{2}$$

Solution

求系数

- 状态为 n^2 没有前途，尝试优化
- 我们发现同时对两侧做 DP 非常蠢，他们实际上是独立的
- 因此只对一侧做，设 g_i 表示右侧有 i 个数字，最后只剩下一个时，第一个数的系数期望
- 转移依旧考虑加一个数只需要考虑最后一个空隙被合并的顺序，
$$g_i = \frac{1}{i} \times \frac{g_{i-1}}{2} + \frac{i-1}{i} \times g_{i-1}$$
- 则 $k_i = g_{i-1} \times g_{n-i}$
- 复杂度瓶颈是求逆元，可以线性求逆元，也可以直接快速幂，均可快速通过

题面

若干序列，每次可以任何一个序列的开头取一个数加入操作序列 opt ，要求最小化

$$\sum_i \sum_{i \leq j} opt_j$$

多次修改，每次修改形如在某个序列开头或结尾添加元素

$$\sum len, k, q \leq 3 \times 10^5$$

Solution

- 先不考虑带修的情况，对式子变形，考虑每个数的贡献，会得到

$$\sum_{i=1}^N (N+1-i) opt_i$$

- 考虑序列内部，若 $x < y$ ，则 x 先取显然更优，即 $2x + y < 2y + x$
- 但有可能 x 排在 y 后面，而规则限制我们必须先取前列的数。因此若 $a_i > a_{i+1}$ ，我们一定会在取完 a_i 之后立刻取 a_{i+1} ，否则我们交换操作顺序一定不劣

Solution

- 因此我们进一步采取交换论证，现在有两个“连续取”段 $(a_1, a_2, \dots, a_n)$ 和 $(b_1, b_2, \dots, b_m)$ 。我们有两种选择：

- 先 a 后 b : $X = \sum_{i=1}^n a_i(n+m+1-i) + \sum_{i=1}^m b_i(m+1-i)$
- 先 b 后 a : $Y = \sum_{i=1}^m b_i(n+m+1-i) + \sum_{i=1}^n a_i(n+1-i)$

- 作差 $X - Y = m \sum a_i - n \sum b_i$ ，再令 $X - Y < 0$ 得到 $\frac{\sum_i a_i}{n} < \frac{\sum b_i}{m}$

- 这表明先 a 后 b 当且仅当 a 的平均数更小

- 再来思考合并连续段需要维护什么信息：

$$X = m \sum_{i=1}^n a_i + \sum_{i=1}^n a_i(n+1-i) + \sum_{i=1}^m b_i(m+1-i)$$

- 可以发现，只需要维护各自的答案、总和和元素个数即可
- 由此我们确定了我们的策略：先不断合并序列内部，直到序列内变为若干连续段，满足平均数不降，再对所有连续段排序，从小到大合并

Solution

- 现在考虑带修，显然在序列前端和后端加数字只会促进段的合并。例如，有可能在开头加了一个较大的元素，这使得一些前缀发生了合并
- 既然只会发生合并，那么连续段的总个数是 $O(n + q)$ 的
- 可以直接用平衡树实时维护有哪些连续段。
- 更好的做法是离线出所有出现过的连续段，记录每个连续段出现与消失的时间。最后把出现过的所有连续段排序，再挂到线段树叶子上，在扫描线的过程中用 `push_up` 维护合并，根节点的值即为答案
- 复杂度为 $O((n + q) \log(n + q))$ ，可以使用 `std::deque` 或者 `std::list` 维护，具体实现可以参考 `std`。

Thanks!